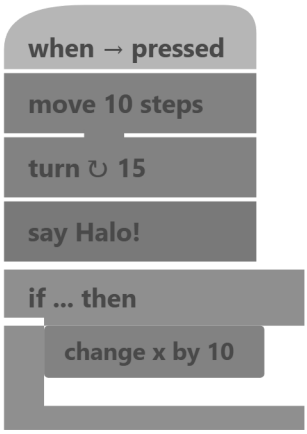
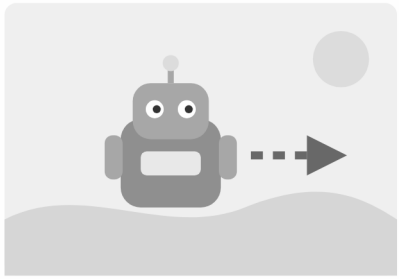
INFORMATIKA KELAS VII · SEMESTER GANJIL

BAB 1
Berpikir Komputasional

Materi Pembelajaran untuk Peserta Didik

PERTEMUAN 3
Algoritma dan Dasar Pemrograman (Scratch)





Urutan · Percabangan · Perulangan

Mata Pelajaran	: Informatika	**Kelas**	: VII
Satuan Pendidikan	: SMP Negeri 6 Mengwi	**Tahun Ajaran**	: 2026/2027
Alokasi Waktu	: 3 × 40 menit		
Guru	: Ni Nyoman Dwi Mustikawati, S.Kom.		

Peta Pembelajaran Bab 1

PERTEMUAN 1 ✓
Berpikir Komputasional dalam Kehidupan Sehari-hari: Algoritma, Optimasi Penjadwalan, Struktur Data, Representasi Data

PERTEMUAN 2 ✓
Literasi Bahasa Indonesia dalam Informatika & Tahapan Berpikir Komputasional

PERTEMUAN 3 ★
Algoritma dan Dasar Pemrograman (Pengenalan Pemrograman & Scratch)

PERTEMUAN 4
Rangkuman Materi Bab 1 & Ulangan Harian Bab 1

Tujuan Pembelajaran Pertemuan 3

Setelah mengikuti pembelajaran ini, kalian diharapkan mampu:

1. Menjelaskan pengertian **pemrograman** dan mengenal lingkungan **Scratch** (area blok, area skrip, panggung, *sprite*).
2. Menjelaskan logika dasar pemrograman: **urutan (sequence)**, **percabangan (selection)**, dan **perulangan (loop)**.
3. Menyusun algoritma sederhana, menerjemahkannya ke blok Scratch, lalu menguji dan memperbaikinya (*debugging*).

Ingat Kembali Pertemuan 2

Dekomposisi
pecah masalah besar

Pengenalan Pola
temukan kesamaan

Abstraksi
saring yang penting

Algoritma
susun langkah logis

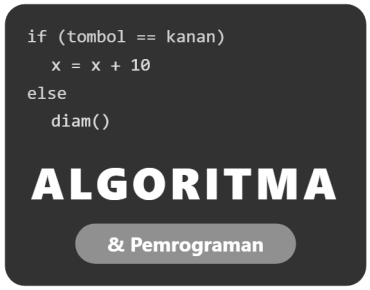
Pertanyaan Pemantik

1. Bagaimana cara melatih pemrograman menggunakan Scratch?
2. Pernahkah kalian bermain game? Menurut kalian, bagaimana komputer “tahu” karakter harus bergerak saat tombol ditekan?
3. Jika hujan, kalian membawa payung; jika tidak, kalian langsung berangkat. Apa hubungannya dengan cara komputer mengambil keputusan?

Pendalaman Materi

B Algoritma dan Dasar Pemrograman

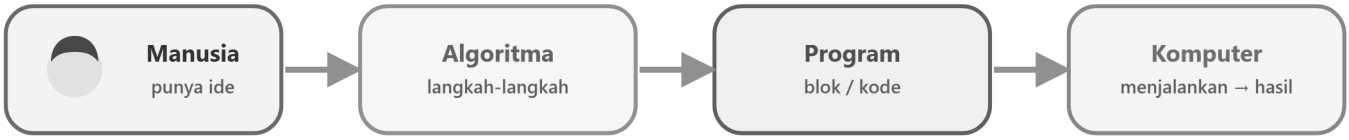
Pemrograman adalah salah satu keterampilan penting di era digital. Dengan memahami dasar-dasar pemrograman, kita belajar cara komputer bekerja dan bagaimana sebuah perintah dapat menghasilkan sesuatu yang bermanfaat.



Bagi pemula, belajar pemrograman bisa dimulai dengan cara yang menyenangkan dan mudah dipahami. Salah satu caranya adalah memakai platform visual seperti **Scratch**. Di sana kita belajar membuat program dengan **menyusun blok-blok perintah** tanpa perlu menulis kode yang rumit.

1. Pengenalan Dasar Pemrograman

Pemrograman adalah cara **memberi perintah kepada komputer** agar dapat menjalankan tugas tertentu. Dengan belajar pemrograman, kita memahami cara kerja teknologi dan mampu menciptakan karya digital sendiri. Pemrograman tidak hanya soal menulis kode, tetapi juga melatih **logika berpikir** dan kemampuan **memecahkan masalah secara sistematis**.



a. Cara Belajar *Programming*

Belajar *programming* adalah proses memahami cara berpikir logis untuk memberi instruksi kepada komputer. Supaya mudah, belajarlah **secara bertahap**, mulai dari konsep dasar hingga mencoba membuat program sederhana. Cara ini sekaligus melatih ketelitian dan kesabaran dalam menyusun langkah kerja komputer.

1
Mulai dari konsep sederhana
Pahami urutan langkah, keputusan, dan pengulangan lewat kegiatan sehari-hari, misalnya langkah memasak atau mengikat tali sepatu.

2
Gunakan bahasa pemrograman visual
Scratch cocok untuk pemula: cukup menyusun blok warna yang mewakili perintah, tanpa menulis kode rumit.

3
Berlatih dengan proyek kecil
Buat animasi sederhana atau permainan interaktif untuk menerapkan logika pemrograman secara nyata.

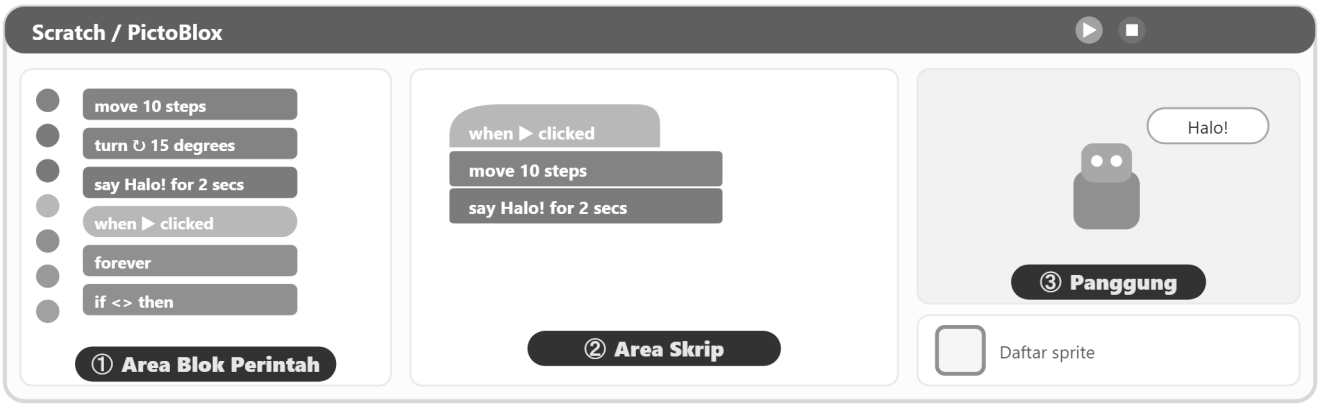
4
Perbaiki kesalahan dengan refleksi
Kesalahan adalah bagian penting dari belajar. Jika program tidak berjalan sesuai harapan, analisis langkahnya dan cari bagian yang salah.

5
Kembangkan kreativitas & kolaborasi
Programming juga tentang menciptakan sesuatu yang bermanfaat. Bekerja sama dengan teman memperkaya ide dan membuat belajar lebih menyenangkan.

b. Pengantar Scratch

Scratch adalah bahasa pemrograman visual yang dirancang khusus agar pemula dapat belajar *coding* dengan menyenangkan. Kita tidak perlu menulis baris kode, tetapi cukup **menyusun blok-blok berwarna yang saling terhubung seperti *puzzle***. Setiap blok punya fungsi tertentu, misalnya menggerakkan karakter, menambahkan suara, atau mengubah tampilan. Dengan tampilan visual ini, kita bisa memahami logika pemrograman tanpa menghafal sintaks atau aturan penulisan.

1 Mengenal lingkungan Scratch



Tampilan Scratch terdiri atas tiga bagian utama: ① **area blok perintah** untuk memilih perintah, ② **area skrip** untuk menyusun blok menjadi algoritma, dan ③ **panggung** untuk melihat hasil program yang dijalankan.

2 Karakter dan kostum (*Sprite*)



Objek utama di Scratch disebut ***sprite***. Sprite bisa berupa karakter, benda, atau gambar yang digerakkan sesuai perintah. Sprite dapat dipilih dari galeri, digambar sendiri, atau diunggah dari perangkat. Sprite juga punya beberapa **kostum** yang bisa diganti untuk membuat animasi atau perubahan gerak.

3 Blok perintah dan fungsinya

Setiap blok memiliki **warna dan bentuk berbeda** sesuai fungsinya. Dengan memahami fungsi tiap blok, kita bisa menyusun instruksi seperti membuat karakter menari, berbicara, atau berinteraksi dengan sprite lain.

	Kategori	Fungsi
●	Motion (gerakan)	bergerak, berputar
●	Looks (tampilan)	bicara, ganti kostum
●	Sound (suara)	memutar suara
●	Events (kejadian)	pemicu: klik ▶, tombol
●	Control (kontrol)	tunggu, ulangi, jika
●	Operators	hitung, bandingkan

*Warna mengikuti Scratch 3.0. Warna dapat sedikit berbeda pada versi atau aplikasi lain.

4 Penyusunan algoritma dalam Scratch



Membuat program di Scratch berarti **menyusun blok perintah secara berurutan** sesuai logika algoritma. Misalnya, agar karakter berjalan ke kanan, susun blok *"when right arrow key pressed"* lalu tambahkan blok *"move 10 steps"*. Urutan penyusunan blok menentukan hasil akhir program.

5 Kreativitas tanpa batas

Cerita interaktif | **Permainan** | **Animasi** | **Komunitas *online***

Scratch tidak hanya mengajarkan logika berpikir, tetapi juga mendorong **kreativitas**. Kita bisa membuat cerita interaktif, permainan sederhana, atau animasi sesuai imajinasi. Scratch juga punya komunitas *online* untuk berbagi karya dan belajar dari orang lain, sehingga kita belajar berpikir seperti *programmer* sekaligus berkarya seperti seniman digital.

Info: Scratch & PictoBlox

Saat praktik di kelas, kita juga dapat menggunakan **PictoBlox**, yaitu aplikasi pemrograman blok yang dibangun berdasarkan Scratch. Blok, sprite, panggung, dan cara menyusun programnya sama, sehingga semua yang kalian pelajari di Scratch bisa langsung dipraktikkan di PictoBlox.

c. Logika Dasar Pemrograman

Logika dasar pemrograman adalah kemampuan berpikir **terstruktur dan sistematis** agar komputer dapat menjalankan perintah sesuai keinginan. Logika menentukan cara program bekerja berdasarkan urutan langkah, kondisi, dan hasil yang diharapkan. Tanpa logika yang baik, program bisa salah atau tidak berjalan dengan benar. Karena itu, memahami logika adalah fondasi penting sebelum mempelajari bahasa pemrograman lebih lanjut.

1 Pengertian logika pemrograman

Logika pemrograman adalah cara berpikir yang **mengikuti aturan dan urutan tertentu** agar komputer memahami perintah yang diberikan. Komputer **tidak dapat menebak maksud manusia**, sehingga setiap langkah harus dijelaskan dengan jelas dan logis. Logika membantu kita merancang alur kerja program yang teratur dari awal hingga akhir.

2 Struktur berpikir dalam pemrograman

Struktur berpikir dalam pemrograman dibangun dari tiga struktur dasar berikut.



Urutan dipakai untuk menjalankan langkah secara berurutan, **percabangan** untuk membuat keputusan, dan **perulangan** untuk mengulang proses tertentu secara otomatis.

3 Kondisi dan keputusan (*If–Then–Else*)

Kondisi adalah unsur penting dalam logika pemrograman. Dengan kondisi, program dapat **"memilih" tindakan** berdasarkan situasi tertentu. Misalnya, **jika** tombol ditekan, **maka** karakter bergerak; **jika tidak**, karakter diam. Struktur *If–Then–Else* digunakan untuk mengatur keputusan semacam ini agar program lebih dinamis dan interaktif.

Dalam kehidupan sehari-hari: *jika* hujan saat pulang sekolah, *maka* siapkan payung; *jika tidak*, langsung pulang.



Contoh blok If–Then–Else di Scratch

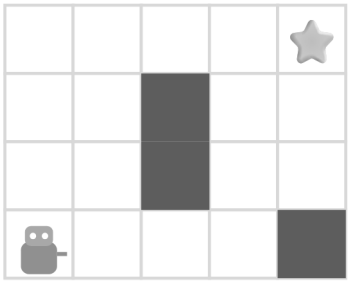
2. Pengembangan Pemrograman

Setelah memahami dasar-dasar pemrograman, langkah berikutnya adalah mengembangkan kemampuan agar dapat membuat program yang lebih terstruktur dan bermanfaat. Kita belajar menyusun algoritma, mengenali pola, dan mengubah ide menjadi karya digital nyata. Pada tahap ini kita berlatih menghubungkan konsep logika dengan media seperti **permainan kartu** dan **aplikasi Scratch**, sehingga kemampuan berpikir komputasional semakin tajam dan kreatif.

a. Pengenalan Kartu Permainan AP (*Unplugged*)

Kartu Permainan AP (*Algorithmic Play*) adalah media belajar **tanpa komputer** untuk memperkenalkan konsep algoritma dengan cara menyenangkan. Melalui permainan ini kita belajar menyusun urutan langkah, membuat keputusan, dan memecahkan masalah secara sistematis. Setiap kartu berisi instruksi yang harus disusun dengan urutan yang benar agar hasilnya sesuai harapan. Kegiatan ini melatih dasar pemrograman tanpa mengetik kode, sekaligus melatih kerja sama, ketelitian, dan berpikir logis.

✦ Ayo Coba! Bantu robot mencapai bintang ⭐



kotak gelap = tembok · robot menghadap kanan

Kartu instruksi:

`↑ Maju 1 kotak` `↰ Belok kiri` `↱ Belok kanan` `🔁 Ulangi ... kali`

Susun kartu agar robot sampai ke bintang tanpa menabrak tembok. Tuliskan urutan kartumu:

.....

.....

.....

💡 Tantangan: bisakah kalian mempersingkat algoritma menggunakan kartu **Ulangi**?

b. Menyusun Langkah-Langkah Algoritma

Dalam pemrograman, algoritma menjadi **dasar utama** dalam menentukan cara komputer menjalankan perintah. Dengan menyusun algoritma, kita belajar berpikir logis dan efisien.

1 Tujuan algoritma

Tujuan algoritma adalah menemukan **cara paling tepat dan cepat** untuk menyelesaikan masalah. Setiap langkah harus **jelas**, **terurut**, dan dapat dijalankan **tanpa menimbulkan kebingungan**.

3 Contoh algoritma sederhana

Program agar karakter berjalan maju:

Mulai › Tekan tombol › Karakter bergerak › Berhenti

Setiap langkah memiliki urutan yang logis dan jelas.

2 Bentuk penyajian algoritma

Algoritma dapat disajikan dalam bentuk **kalimat langkah-langkah**, ***pseudocode***, atau **diagram alur (*flowchart*)**. Bentuk visual seperti *flowchart* membantu kita memahami urutan proses dengan lebih mudah. Contoh: algoritma "karakter bergerak saat tombol panah kanan ditekan".

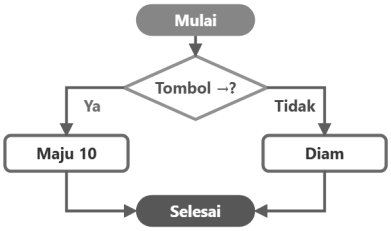
Kalimat langkah

1. Mulai.
2. Periksa apakah tombol panah kanan ditekan.
3. Jika ya, karakter maju 10 langkah.
4. Jika tidak, karakter diam.
5. Selesai.

Pseudocode

```
MULAI
JIKA tombol_kanan DITEKAN
   MAJU 10 langkah
SELAIN ITU
   DIAM
SELESAI
```

Flowchart



c. Implementasi Algoritma dalam Scratch

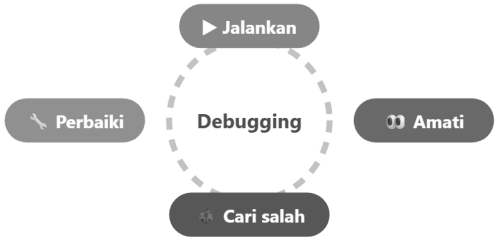
Setelah memahami cara menyusun algoritma, langkah selanjutnya adalah menerapkannya dalam program menggunakan Scratch. Di Scratch, setiap langkah algoritma **diterjemahkan ke dalam blok-blok perintah visual**.

1 Menerjemahkan langkah ke dalam blok Scratch



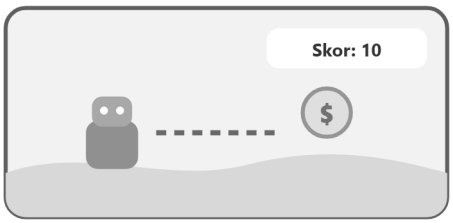
Algoritma *"jika tombol panah ditekan maka karakter bergerak maju"* dibuat di Scratch dengan blok **when right arrow key pressed** diikuti **move 10 steps**. Dengan cara ini kita bisa melihat bagaimana algoritma bekerja secara nyata.

2 Menguji dan memperbaiki program (*debugging*)



Setelah menyusun blok, **jalankan program** untuk melihat hasilnya. Jika hasil belum sesuai, ubah atau perbaiki urutan blok. Proses ini disebut ***debugging*** dan sangat penting dalam pemrograman. Kuncinya: **mencoba dan memperbaiki berdasarkan hasil**.

3 Mengembangkan proyek kreatif

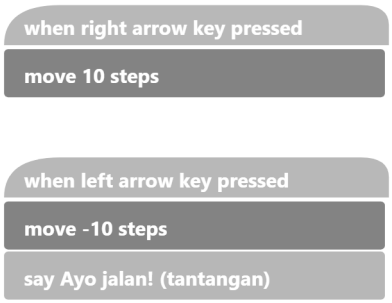


Setelah menguasai cara menerapkan algoritma, cobalah membuat proyek yang **lebih kompleks**, seperti permainan interaktif atau animasi sederhana. Melalui proyek ini kita belajar menggabungkan **logika, seni, dan kreativitas** menjadi satu karya yang menarik.

Ayo Praktik: Karakter Bergerak dengan Tombol Panah

Kerjakan secara berpasangan di **Scratch** atau **PictoBlox**!

1. Buka Scratch/PictoBlox, lalu pilih atau tambahkan satu **sprite**.
2. Dari kategori **Events**, seret blok *when right arrow key pressed* ke area skrip.
3. Dari kategori **Motion**, sambungkan blok *move 10 steps* di bawahnya.
4. Buat skrip kedua: *when left arrow key pressed* + *move -10 steps*.
5. Tekan tombol panah di keyboard dan amati sprite di **panggung**.
6. Jika sprite tidak bergerak sesuai rencana, periksa urutan blok dan perbaiki (***debugging***).
7. **Tantangan:** tambahkan blok *say* agar sprite berbicara saat bergerak!



Rangkuman Pertemuan 3

💻 Dasar Pemrograman

- **Pemrograman** = memberi perintah kepada komputer agar menjalankan tugas tertentu.
- Belajar bertahap: konsep sederhana → bahasa visual → proyek kecil → refleksi → kreativitas.
- **Scratch**: bahasa pemrograman visual berbasis blok; terdiri atas area blok, area skrip, dan panggung. Objeknya disebut **sprite**.
- Komputer **tidak dapat menebak**, sehingga setiap langkah harus jelas dan logis.

🧠 Logika & Pengembangan

Istilah	Arti
Sequence	urutan langkah
Selection	percabangan / keputusan (if–then–else)
Loop	perulangan otomatis
Debugging	mencari & memperbaiki kesalahan program

Algoritma disajikan sebagai **kalimat, pseudocode, atau flowchart**, lalu **diterjemahkan menjadi program** berupa blok Scratch.

Cek Pemahaman

A. Benar atau Salah? Lingkari B (Benar) atau S (Salah)!

1. Di Scratch, kita harus menulis baris kode yang panjang untuk membuat program. **B / S**
2. Objek utama yang digerakkan di Scratch disebut *sprite*. **B / S**
3. Komputer dapat menebak maksud manusia walaupun perintahnya tidak jelas. **B / S**
4. Struktur *If–Then–Else* digunakan agar program dapat memilih tindakan berdasarkan kondisi. **B / S**
5. Kartu Permainan AP adalah media belajar algoritma tanpa komputer. **B / S**

B. Pilihan Ganda: Pilihlah jawaban yang paling tepat!

6. Bagian Scratch yang digunakan untuk melihat hasil program yang dijalankan adalah ....
   A. area blok
   B. area skrip
   C. panggung
   D. daftar kostum
7. Rani menyusun blok *"when space key pressed"* lalu *"say Halo!"*. Kegiatan Rani menunjukkan ....
   A. menghapus algoritma
   B. menerjemahkan algoritma menjadi program
   C. mengganti sprite
   D. membuat flowchart
8. "Ulangi gerakan sebanyak 10 kali" merupakan contoh struktur ....
   A. sequence
   B. selection
   C. loop
   D. debugging
9. Program Budi tidak berjalan sesuai harapan, lalu ia memeriksa dan memperbaiki susunan blok. Proses ini disebut ....
   A. uploading
   B. debugging
   C. browsing
   D. sorting

10. Bentuk penyajian algoritma yang menggunakan simbol dan tanda panah disebut ....

A. pseudocode
B. kalimat langkah
C. flowchart
D. sprite

Refleksi Diri

Pernyataan	Ya	Belum
Saya mengenal bagian-bagian tampilan Scratch.	☐	☐
Saya dapat membedakan sequence, selection, dan loop.	☐	☐
Saya dapat membuat sprite bergerak dengan tombol.	☐	☐
Saya tidak menyerah saat program salah (debugging).	☐	☐

Hal baru yang paling berkesan hari ini:

.....

.....

Pertemuan Berikutnya

Pada Pertemuan 4 kita akan:

1. Mengulas **Rangkuman Materi Bab 1** (Pertemuan 1–3)
2. Melaksanakan **Ulangan Harian Bab 1**

Pelajari kembali materi Pertemuan 1, 2, dan 3 di rumah ya!



Semangat!